

I'm perfectly sane btw

Cơ chế - Tính toán của thuật toán xác định điểm Trends

(ver 1.00)

1. Cơ chế

+) Thuật toán này yêu cầu 4 yếu tố tất cả:

Số bidders tham gia trên khoảng thời gian (Frequency),

Độ tăng của giá tiền (Growth),

Độ xuất hiện của Bots (AutoBidders)

và Áp lực thời gian (TimePressure)

+) Công thức tính điểm:

TrendScore =

$$\frac{((0.40 * \text{Growth} * \text{GC} + 0.35 * \text{Freq} * \text{FC} + 0.25 * \text{AutoBid} * \text{AtBC})}{(\text{Tổng các trọng số} * \text{hệ số})) * \text{TimePressure}}$$

Trọng số: 0.4, 0.35, 0.25 có thể thay đổi theo ý muốn của Admin/Sellers, nên đảm bảo tổng = 1, điều này có thể quyết định yếu tố nào quan trọng hơn

Hệ số: GC, FC, AtBC đóng vai trò như hình phạt / phần thưởng điểm cho các yếu tố trên

2. Cách xác định và tính toán

+) FrequencyAnalyzer:

Yêu cầu: Tổng thời gian của phiên đấu

Tổng các lượt đặt trong 1/10 tổng thời gian vừa rồi và trước đó

(trường hợp trước đó không có, coi đó là lượt tăng tốc đầu tiên: $\text{Freq} = \text{FC} = 1$)

Tính toán: Lấy tỉ lệ tổng lượt đặt vừa rồi / Tổng lượt đặt trước đó

```
// FIX: Cap ratio ở 2.0 để tránh phá vỡ thang điểm
BigDecimal cappedRatio = ratio.min(new BigDecimal(val: "2.0"));

String status;
Rename usages
BigDecimal freqCoeff;

if (recentBids > lastRecentBids) {
    status = "Tăng tốc";
    freqCoeff = new BigDecimal(val: "1.5");
} else if (recentBids < lastRecentBids) {
    status = "Giảm tốc";
    freqCoeff = new BigDecimal(val: "0.8");
} else {
    status = "Không đổi";
    freqCoeff = BigDecimal.ONE;
}
```

+) GrowthAnalyzer:

Yêu cầu: Transaction lấy lượt đặt mới nhất và gần mới nhất

(trường hợp trước đó không có, coi đó là lượt đầu tiên tương tự trên)

Tính toán: Lấy độ tăng: (Giá hiện tại – Giá trước) / Giá trước. Căn cứ theo độ tăng, gán các giá trị tương ứng

(Không trực tiếp lấy luôn tỉ lệ do chênh lệch quá lớn)

```
BigDecimal growthPercent = currentPrice.subtract(previousPrice)
    .divide(previousPrice, scale: 4, RoundingMode.HALF_UP);
double growth = growthPercent.doubleValue();

BigDecimal assignedValue;
BigDecimal priceCoefficient;
String status;

if (growth <= 0.05) {
    assignedValue = new BigDecimal(val: "0.5");
    priceCoefficient = new BigDecimal(val: "0.8");
    status = "Tăng chậm";
} else if (growth <= 0.2) {
    assignedValue = new BigDecimal(val: "1.0");
    priceCoefficient = BigDecimal.ONE;
    status = "Tăng đều";
} else if (growth <= 1) {
    assignedValue = new BigDecimal(val: "1.5");
    priceCoefficient = new BigDecimal(val: "1.2");
    status = "Bứt phá";
} else {
    assignedValue = new BigDecimal(val: "2.0");
    priceCoefficient = new BigDecimal(val: "1.5");
    status = "Tăng đột biến";
}
```

+) TimePressureAnalyzer:

Yêu cầu: Tổng thời gian của phiên đấu và Tổng thời gian còn lại của phiên

Tính toán: Lấy tỉ lệ Thời gian còn lại / Tổng thời gian, sau đó áp dụng công thức

$$\text{pressure} = 1.0 + 0.25 * (1.0 - \text{timeLeftRatio});$$

(Phần tính tổng thời gian còn lại dùng `now()` ở thời điểm kích hoạt xu hướng (WIP))

```
long totalSeconds = Duration.between(
    context.auction().getStartTime(),
    context.auction().getEndTime()
).toSeconds();

long secondsLeft = Duration.between(
    context.now(),
    context.auction().getEndTime()
).toSeconds();

double timeLeftRatio = (double) secondsLeft / totalSeconds;

// Time Pressure = 1.0 + 0.25 * (1 - timeLeftRatio)

double pressure = 1.0 + 0.25 * (1.0 - timeLeftRatio);

String label;
if (timeLeftRatio >= 0.5) {
    label = "Chưa vội";
}

else if (timeLeftRatio >= 0.1) {
    label = "Sắp hết";
}

else {
    label = "Vội";
}
```

+) AtBAnalyzer:

Yêu cầu: Số bots kích hoạt trong phiên và Số bidders trong phiên

Tính toán: Lấy tỉ lệ Số bots / Số bidders

```
if (autobidEnabledCount == 0) {  
    // Phiên sạch: không có auto-bid  
    return new TrendSignal(  
        new BigDecimal(val: "1.0"),  
        BigDecimal.ONE,  
        label: "Phiên sạch",  
        reason: "Không có auto-bid"  
    );  
}  
  
// Xác định mức độ bots trong phiên:  
// Số bots trong phiên(lấy từ auto_bid_config) / Số bidders  
  
BigDecimal ratio = new BigDecimal(autobidEnabledCount)  
    .divide(new BigDecimal(bidderCount), scale: 4, RoundingMode.HALF_UP);  
  
if (ratio.compareTo(new BigDecimal(val: "0.5")) < 0) {  
  
    // Có auto-bid ở mức bình thường(<50%)  
    return new TrendSignal(  
        ratio,  
        new BigDecimal(val: "0.8"),  
        label: "Có auto-bid",  
        reason: autobidEnabledCount + " auto-bid đang bật"  
    );  
} else {  
    // Bot war: phạt điểm thấp(>50%)  
    return new TrendSignal(  
        ratio,  
        new BigDecimal(val: "0.5"),  
        label: "Bot war",  
        reason: autobidEnabledCount + " auto-bid đang bật"  
    );  
}
```

+) Service:

- Đa luồng trong Service

```
// Tạo tất cả CompletableFuture cùng lúc - không chờ từng cái
List<CompletableFuture<Optional<AuctionTrendResponse>>> futures =
    summaries.stream().map(summary -> CompletableFuture.supplyAsync(() -> buildTrend(summary)))
    .collect(toList());

// Chờ tất cả xong 1 lần
List<AuctionTrendResponse> trends =
    futures.stream().map(CompletableFuture::join)
    .flatMap(Optional::stream)
    .sorted(
        Comparator.comparing(
            AuctionTrendResponse::trendScore,
            Comparator.nullsLast(Comparator.reverseOrder())
        ))
    .collect(Collectors.toList());

return new Response(success: true, message: "Tải xu hướng đầu giá thành công", trends);
} catch (Exception e) {
    e.printStackTrace();
    return new Response(success: false, message: "Lỗi máy chủ khi phân tích xu hướng: " + e.getMessage(), data: null);
}
```

Dòng 3: Chạy qua các summaries qua hàm buildTrend được chạy bất đồng bộ, để thành stream (luồng) của Optional<AuctionTrend> và đổi về List<CompletableFuture<...>> (Dòng 4)

Dòng 7: Chạy qua các futures bên trên, sau đó kiểm tra các Optional<Summary> (Dòng 8), nếu không null trả về để sort (Cơ chế khá giống các if (Optional != null) của các bên)

.map: Khi này các CompletableFuture kia đã xong(.join()), thu về Option<AuctionTrend>

.flatMap: Giúp ta gỡ bọc Optional<>, qua đó trả về dạng AuctionTrend

.sorted: So sánh theo thứ tự điểm trend giảm dần

.collect: Cho phép tập hợp các đối tượng AuctionTrend

(Gemini AI tổng hợp phần sau (Dòng5 ->))

List<Summary> $\xrightarrow{\text{Dòng 3-4}}$ List<CompletableFuture> $\xrightarrow{\text{Dòng 9 (join)}}$

Stream<Optional> $\xrightarrow{\text{Dòng 10 (flatMap)}}$ Stream<AuctionTrend> $\xrightarrow{\text{Dòng 11-17}}$ List<AuctionTrend> (Đã sort)

- Tính toán trong Service

```
TrendSignal freqSignal = frequencyAnalyzer.analyze(context);
TrendSignal autoBidSignal = autoBiddersAnalyzer.analyze(context);
TrendSignal growthSignal = priceGrowthAnalyzer.analyze(context);
TrendSignal timeSignal = timePressureAnalyzer.analyze(context);

BigDecimal growthVal = growthSignal.value();
BigDecimal freqVal = freqSignal.value();
BigDecimal autoBidVal = autoBidSignal.value();
BigDecimal timePressureMultiplier = timeSignal.value();

BigDecimal freqCoeff = freqSignal.coefficient();
BigDecimal autoBidCoeff = autoBidSignal.coefficient();
BigDecimal growthCoeff = growthSignal.coefficient();

// BaseScore = (0.40 * Growth * GC + 0.35 * Freq * FC + 0.25 * AutoBid * AtBC)
BigDecimal baseScore = growthVal.multiply(growthWeightValue).multiply(growthCoeff)
    .add(freqVal.multiply(freqWeightValue).multiply(freqCoeff))
    .add(autoBidVal.multiply(autoBidWeightValue).multiply(autoBidCoeff));

// FinalScore = BaseScore * TimePressure / (sigma(Weight * Coefficient))
BigDecimal finalScore = baseScore.multiply(timePressureMultiplier)
    .divide( (growthWeightValue.multiply(growthCoeff)
        .add(freqWeightValue.multiply(freqCoeff))
        .add(autoBidWeightValue.multiply(autoBidCoeff))), scale: 4, RoundingMode.HALF_UP)
    .setScale( newScale: 4, RoundingMode.HALF_UP);

// Debug
System.out.printf("[Trend] %s | G=%.4f F=%.4f A=%.4f TP=%.4f → Score=%.4f\n",
    summary.auctionId(),
    growthVal.doubleValue(), freqVal.doubleValue(),
    autoBidVal.doubleValue(), timePressureMultiplier.doubleValue(),
    finalScore.doubleValue());
```

Cũng giống như ý định đầu Insight

3. FAQ

a) Ở thế trước bảo là sẽ căn cứ theo độ dốc đồ thị mà, bíp à?

- Cơ bản thì phương pháp trên sẽ đa phần để mô phỏng thôi, đồng thời căn cứ trên 4 yếu tố trên thì tôi có thể nói là tương đối chắc chắn, tuy vậy lúc nào nhóm mình kiểm tra lần cuối mới được

b) Code nhìn AI vãi

- Thì đúng, idea thì tôi nghĩ, chứ cú pháp cùng nhận xét, chỉnh sửa thuật toán thì tôi vẫn để cho Claude xử lý. Nhưng yên tâm là tôi luôn nhớ và hiểu cái thuật toán này sau gần 24h ngồi trước màn hình mà phân tích

c) Code nhìn hơi smell đấy

- Cơ bản như lần trước tôi có bảo thì dự định ban đầu là ghép vào bảng chính luôn, nhưng phần là lười và các bác bảo để thêm 1 bảng chuyên dụng nên tôi vẫn để đấy, không đụng chạm quá nhiều đâu

4. Plans

- ~~— Cho phép hiện phiên cao điểm nhất trên Dashboard~~
- ~~— Test xong cần chỉnh sửa + xin ý kiến cải thiện thuật toán~~
- ~~— Quan trọng nhất: Kiểm tra khả năng Realtime và Announcer do việc bắt đầu tính chỉ khi gọi khiến cho chương trình chạy quá lâu.~~
- ~~— Giải thích cụ thể hơn cách hoạt động của phần đa luồng (48-70)~~
- ~~— Cuối cùng là hạ cái tôi xuống và ngủ đúng giờ.~~

5. Final Thoughts

Thì cái nên chú ý vẫn là phần đa luồng và các yếu tố được dùng để xác định, tính toán cho thuật toán. Ideas còn sơ khai nhưng theo cách nào đấy vẫn xử lý được. Nếu còn thời gian để rework/review/recreate, dự án sẽ được cải thiện nhiều thứ, trong số đó là siêu tổng hợp kiến thức chẳng hạn – Insighters.

For the Brotherhood.

Cosx

- 🤔

3 Đẩy cấu hình ra bên ngoài (External Configuration):

- Đưa toàn bộ các trọng số (0.4 , 0.35 , 0.25) và các hệ số phạt/thưởng vào file cấu hình `application.yml` hoặc lưu trữ trong một bảng cấu hình hệ thống dưới Database. Việc này cho phép Seller hoặc Admin thay đổi chiến lược tính điểm xu hướng ngay lập tức thông qua giao diện Quản trị mà không cần can thiệp vào mã nguồn. PDF + 1